

Project Hackathon: ChronoSync – The Temporal nexus

Duration: 48 Hours

Stack: React (Frontend) + Express.js with db.json (Backend)

Overview

Your society is building **ChronoSync**, a secure, time-bound task coordination system for a distributed crew. Tasks in this system have a strict expiration timestamp; if they are not boosted or completed before time runs out, they expire.

You will implement local persistent JSON storage (db.json) and secure, stateful server-side **Session Management** using cookies.

Architectural Constraints

- **The File-Based Database:** You will use db.json managed by standard Node.js File System operations (fs/promises) or a light utility wrapper like lowdb. Do not use MongoDB, PostgreSQL, or SQLite.
- **Stateful Security:** You must use HTTP-Only session cookies to track authenticated sessions. No manual header tokens or localStorage token storage allowed.
- **CORS & Credentials:** Since you are passing cookies between a React app and an Express app running on different ports, you must configure CORS with credentials: true on both ends.

Backend Requirements (Express.js)

Your server must run on <http://localhost:5000> and use the express-session library.

1. Database Schema (db.json)

Your JSON file must store two distinct collections: users and tasks.

```
{
  "users": [
    {
```

```

    "username": "commander",
    "password": "hashed_or_plain_password",
    "clearance": "LEVEL_3"
  }
],
"tasks": [
  {
    "id": "task_102",
    "description": "Recalibrate life support telemetry matrix.",
    "clearanceRequired": "LEVEL_3",
    "expirationTimestamp": 1785945600000
  }
]
}

```

2. The Authentication & Session Lifecycle

Session Initialization

Configure express-session with:

- **secret:** A secure string key.
- **cookie.httpOnly:** true: Prevents client-side JavaScript from reading the session cookie (protecting against XSS).
- **cookie.sameSite:** 'lax' (or 'none' if testing across secure origins).

Middleware A: Session Clearance Guard

- **Behavior:** Applies to all mutation routes (POST, DELETE).
- **Checks:** Inspects req.session.user. If missing, immediately return a 401 Unauthorized status code. It must also match the clearance level of the user against the clearanceRequired field of the targeted task. If the user's level is insufficient, return 403 Forbidden.

3. API Endpoints

- **POST /api/auth/login**
 - Accepts username and password. Verifies against the users array in db.json.
 - If valid, attaches the user object (minus password) to req.session.user and returns 200 OK.
- **POST /api/auth/logout**
 - Destroys the server session using req.session.destroy() and clears the client cookie.
- **GET /api/nexus/tasks**
 - Reads from db.json.

- **Server-Side Security Filter:** Before sending the array back, the server must filter out and delete any records from db.json where expirationTimestamp is less than Date.now().
- **POST /api/nexus/tasks**
 - *Protected by Clearance Guard.*
 - Accepts a new task description and required clearance. Appends it to db.json using atomic file writes. Automatically sets expirationTimestamp to exactly 10 minutes into the future.
- **DELETE /api/nexus/tasks/:id**
 - *Protected by Clearance Guard.*
 - Deletes the task matching the ID from db.json.

Frontend Requirements (React)

Your frontend layout must dynamically adjust its views depending on the active user session status.

1. Session-Aware Network Configuration

- **Crucial Step:** Configure your HTTP client (axios instance or your custom fetch wrapper) to include credentials: 'include' on every request. If you forget this, the browser will block session cookies from being transmitted to your Express server.

2. Unified Auth Portal / Gateway

- If no active session is found on application load, present a clean, dark-themed login terminal interface hiding the main board.
- Upon a successful 200 OK from the login route, update a global React state (user) to trigger a smooth transitions into the main operation panel.

3. Clearance-Based UI Controls

- Render all active tasks on a central dashboard grid.
- Each card must feature a live countdown timer counting down against the expirationTimestamp.
- **Conditional Rendering:** If the logged-in user's clearance level is lower than the card's clearanceRequired attribute, the "Delete" action button must be disabled or entirely hidden in the UI.

Evaluation Criteria

Submissions will be rigorously verified against bypass attempts within the 48-hour timeline:

1. **Persistence Resilience:** If the Express server is completely shut down and restarted mid-evaluation, do the active tasks and registered users remain persistent inside db.json?
2. **Session Security Isolation:** If an evaluator tries to interact with the API endpoints using an external client like Postman without authenticating first via the login route, does the session middleware successfully reject them?
3. **Cross-Origin Cookie Integrity:** Did the team successfully manage the handshake between React and Express ports using proper CORS credential flags without relying on unsafe header overrides?
4. **Clean File Operations:** Are file mutations on db.json properly asynchronous (fs.promises) preventing blockages on the Express event loop?
5. **UI/UX:** The design should look like a modern system interface. All the interactions and transitions must be smooth. No unnecessary global rerenders.